SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task Set 1

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1501

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I **Pugazharasan K (192511341)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Pugazharasan K**

QUESTION 1

A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer

The likely design error is poor input-split sizing or an unbalanced partitioning strategy. Very large or uneven input splits can cause some mapper tasks to process much more data than others. If the map-side workload is not balanced, reducers may also receive little or no data, especially when the mapper output is small or the number of reducers is unnecessarily high.

Likely Causes

- Input files or split sizes are highly uneven, creating long-running mapper tasks.
- The configured split size is not appropriate for the input dataset.
- The number of reducers is too high compared with the amount of intermediate data.
- The partitioning key is skewed, so some reducers receive much more data while others remain idle.
- A poor custom partitioner may distribute keys unevenly.

Corrective Actions

- Review input file sizes and configure an appropriate input split/block size so mapper workloads are more balanced.
- Avoid a large number of very small files; combine small files where appropriate.
- Choose the reducer count according to the actual intermediate-data volume and available cluster resources.
- Use a balanced partitioning strategy so keys are distributed more evenly among reducers.
- Investigate key skew using counters, task-level input/output statistics, and reducer input sizes.
- Use a combiner when the operation supports it, reducing unnecessary shuffle data.
- Monitor mapper and reducer execution times and verify that no task is acting as a straggler.

QUESTION 2

An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer

The most probable error is insufficient or incorrectly placed HDFS replicas, possibly combined with a single point of failure in the NameNode. If a block has only one copy, losing its DataNode makes that block unavailable. Even with multiple replicas, poor placement can make several copies vulnerable to the same rack or infrastructure failure.

Probable Design Errors

- Replication factor is set too low, such as 1, for data that requires fault tolerance.
- Some files were created with an unexpectedly low replication factor.
- Replicas are not distributed across independent racks or failure domains.
- The NameNode is deployed as a single non-HA service, creating a metadata-service single point of failure.
- DataNode health and under-replicated blocks are not being monitored or repaired promptly.

Corrective Actions

- Use an appropriate replication factor, commonly 3 for important HDFS data, while considering storage cost.
- Verify the actual replication factor of critical files and correct under-replicated blocks.
- Enable rack-aware replica placement so copies are distributed across racks.
- Deploy NameNode high availability with an active and standby NameNode and automatic failover.
- Monitor missing, corrupt, and under-replicated blocks and allow HDFS to re-replicate lost copies.
- Test node-failure recovery periodically to verify that data remains accessible.

QUESTION 3

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer

Duplicate records usually indicate that the ingestion pipeline does not have reliable delivery tracking or idempotent loading. Repeated retries, replayed files, overlapping extraction windows, or missing unique-record checks can cause the same source data to be inserted more than once.

Likely Causes

- NiFi retries a flow after a failure but the downstream store has already committed the previous copy.
- The source extraction repeatedly reads the same records because there is no reliable incremental-load condition.
- A file is reprocessed after a restart because its processing state was not persisted or checked.
- The ETL pipeline lacks a unique business key or record identifier.
- Multiple ingestion paths write the same source data to the analytics store.
- The destination uses append-only loading without duplicate detection or upsert/merge logic.

Recommended Fixes

- Assign or preserve a stable unique record ID or business key and enforce uniqueness at the destination.
- Use idempotent loading so processing the same input again produces the same final state rather than another copy.
- Use incremental extraction based on timestamps, change markers, sequence numbers, or source-supported CDC mechanisms.
- Track processed files/records using metadata, provenance, or an ingestion-control table.
- Configure NiFi retry and failure relationships carefully so a failed delivery is not blindly duplicated after a successful commit.
- Use checksums or file identifiers to detect repeated file ingestion where appropriate.
- Audit all ingestion paths and remove overlapping ETL jobs.

QUESTION 4

A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer

The likely scheduling error is that resources are not divided fairly among users or application classes. One or a few applications may be consuming most cluster resources, leaving insufficient capacity for other jobs. Missing queue limits, poor capacity allocation, or an unsuitable scheduling policy can cause starvation.

Likely Scheduling Errors

- No meaningful resource queues or user/application limits are configured.
- A few large jobs request most of the cluster resources.
- The scheduler is not configured to provide fair sharing among competing workloads.
- Queue capacities or maximum resource limits are poorly chosen.
- The system lacks application priorities or resource reservations for critical workloads.

Better Resource Management Approach

- Create separate YARN queues for different users, teams, or workload classes.
- Use a fair-sharing policy so active users receive reasonable access to cluster resources.
- Set minimum capacities for important workloads and maximum capacities to prevent one queue from monopolizing the cluster.
- Configure application/user limits so a single user cannot consume all available containers.
- Use priorities for genuinely critical workloads while maintaining fairness for other users.
- Monitor queue utilization, pending applications, container allocation, and resource utilization.
- Allow unused resources to be temporarily shared with other queues so overall cluster utilization remains high.

QUESTION 5

A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer

The probable flaw is that the recovery process is restoring from a stale backup or snapshot rather than the most recent valid data. Replication and backup are also being treated as if they provide the same protection, even though they solve different failure scenarios.

Probable Design Flaws

- Backups or snapshots are taken too infrequently, producing a large recovery-point gap.
- The restore procedure selects an older backup instead of the latest verified backup.
- Backup metadata, timestamps, or version information are incomplete or incorrect.
- Replication protects against node failure but does not protect against accidental deletion or corruption if the same change is replicated to all copies.
- Backup copies are not independently verified through restoration tests.
- There is no clearly defined Recovery Point Objective (RPO) and Recovery Time Objective (RTO).

Corrective Actions

- Define an RPO that specifies how much recent data loss is acceptable and schedule backups accordingly.
- Maintain versioned backups or snapshots so multiple recovery points are available.
- Select the newest verified backup that satisfies the recovery requirement.
- Keep backup metadata including creation time, dataset version, source, and integrity status.
- Separate backup storage from primary replicated storage so a logical error does not immediately affect every copy.
- Perform regular restore tests to confirm that backups are usable and current.
- Use checksums or integrity verification to detect corrupted backup data.